

Relatório de Arquitetura: Mini E-commerce Distribuído

1. Comunicação entre os Microsserviços

A comunicação entre os microsserviços foi implementada de forma síncrona utilizando o padrão **HTTP/REST**. Para isso, foi utilizada a biblioteca assíncrona `httpx` no ambiente Python.

A arquitetura adota uma separação clara de tráfego: as requisições externas (Tráfego Norte-Sul) passam pelo API Gateway, enquanto a comunicação inter-serviços (Tráfego Leste-Oeste) ocorre de forma isolada na rede interna provida pelo Docker. Por exemplo, quando o Serviço de Pedidos precisa validar a existência e o preço de um item, ele faz uma requisição HTTP `GET` direta para a URL interna do Serviço de Produtos (ex:

`http://ms-products:5002/products/{id}`), sem repassar a requisição pelo Gateway, evitando assim latência desnecessária e gargalos no roteador principal.

2. Estratégia de Consistência na Replicação

Foi adotada a **Consistência Forte** na replicação do Serviço de Produtos.

No momento em que a API recebe uma requisição `POST` para a criação de um novo produto, a aplicação abre e escreve os dados simultaneamente em ambos os arquivos de armazenamento (`db_replica1.json` e `db_replica2.json`). O sistema apenas retorna o status de sucesso (`HTTP 201 Created`) para o cliente após garantir que a escrita foi concluída nas duas réplicas.

Essa escolha foi feita em função da estratégia de leitura adotada (Round-Robin). Como requisições de leitura subsequentes alternam o arquivo consultado a cada chamada, a consistência eventual poderia gerar um cenário em que um cliente cria um produto e, ao tentar visualizá-lo imediatamente em seguida, recebe um erro `404 Not Found` caso a leitura caísse na réplica ainda não sincronizada. A consistência forte elimina esse risco de anomalia de leitura.

3. Tolerância a Falhas no Serviço de Pedidos

O sistema foi desenhado com tolerância parcial a falhas. Caso o Serviço de Pedidos caia (fique offline), o restante do e-commerce **continua funcionando perfeitamente**.

O mecanismo de detecção por *Heartbeat* do API Gateway identificará a ausência de resposta da rota `/health` do Serviço de Pedidos e passará a retornar proativamente o erro `503 Service Unavailable` apenas para as chamadas direcionadas à rota `/orders`. Os serviços de Usuários (registro e autenticação) e Produtos (listagem e criação) permanecem totalmente operacionais e isolados da falha, garantindo que os clientes continuem navegando pela vitrine e criando contas na plataforma enquanto a equipe de engenharia restabelece o checkout.

4. Segurança de Acesso e Escopos via JWT

A restrição de criação de produtos para usuários comuns é garantida pela validação de escopos (*claims*) embutida no payload do token JWT.

Durante o fluxo de login no Serviço de Usuários, o token é gerado com uma chave criptográfica (`SECRET_KEY`) e inclui um atributo chamado `role`, que pode assumir o valor `user` ou `admin`. O Serviço de Produtos possui uma camada de interceptação via injeção de dependência que exige o cabeçalho de autorização. Antes de processar a criação de um produto, este serviço decodifica o JWT usando a mesma chave secreta compartilhada e inspeciona o campo `role`. Se a `role` não for estritamente igual a `admin`, a requisição é abortada de imediato com um erro `403 Forbidden`, impedindo qualquer alteração na base de dados.

5. Limitações em Relação a um Sistema Real de Produção

Apesar de aplicar conceitos arquiteturais corretos, esta implementação acadêmica possui limitações inerentes ao escopo simplificado:

- **Armazenamento Não-Transacional:** A utilização de arquivos `.json` simulando bancos de dados não suporta controle de concorrência ou propriedades ACID. Sistemas em produção utilizam bancos de dados robustos (relacionais ou NoSQL) para evitar corrompimento de dados e *race conditions* sob alto volume de requisições simultâneas.
- **Comunicação Síncrona Fragilizada:** A comunicação HTTP/REST direta entre Pedidos e Produtos pode gerar um efeito cascata em caso de lentidão (gargalo de latência). Sistemas reais frequentemente mitigam isso através de comunicação assíncrona (filas/tópicos com RabbitMQ ou Kafka) ou utilizam padrões como *Circuit Breaker* para evitar travamentos em cadeia.
- **Replicação em Camada de Aplicação:** O controle de replicação e o balanceamento *Round-Robin* foram implementados diretamente no código da aplicação (`main.py`). Em ambientes reais, o balanceamento de carga é delegado à infraestrutura (como Nginx, Kubernetes Services ou Load Balancers na nuvem), e a replicação é gerida nativamente pelo motor de banco de dados escolhido.
- **Segurança de Rede Interna:** A comunicação interna entre os microsserviços ocorre tráfegando texto plano via HTTP. Uma arquitetura de produção que segue o princípio *Zero Trust* exige criptografia TLS ponta a ponta (mTLS) mesmo dentro de redes virtuais privadas.